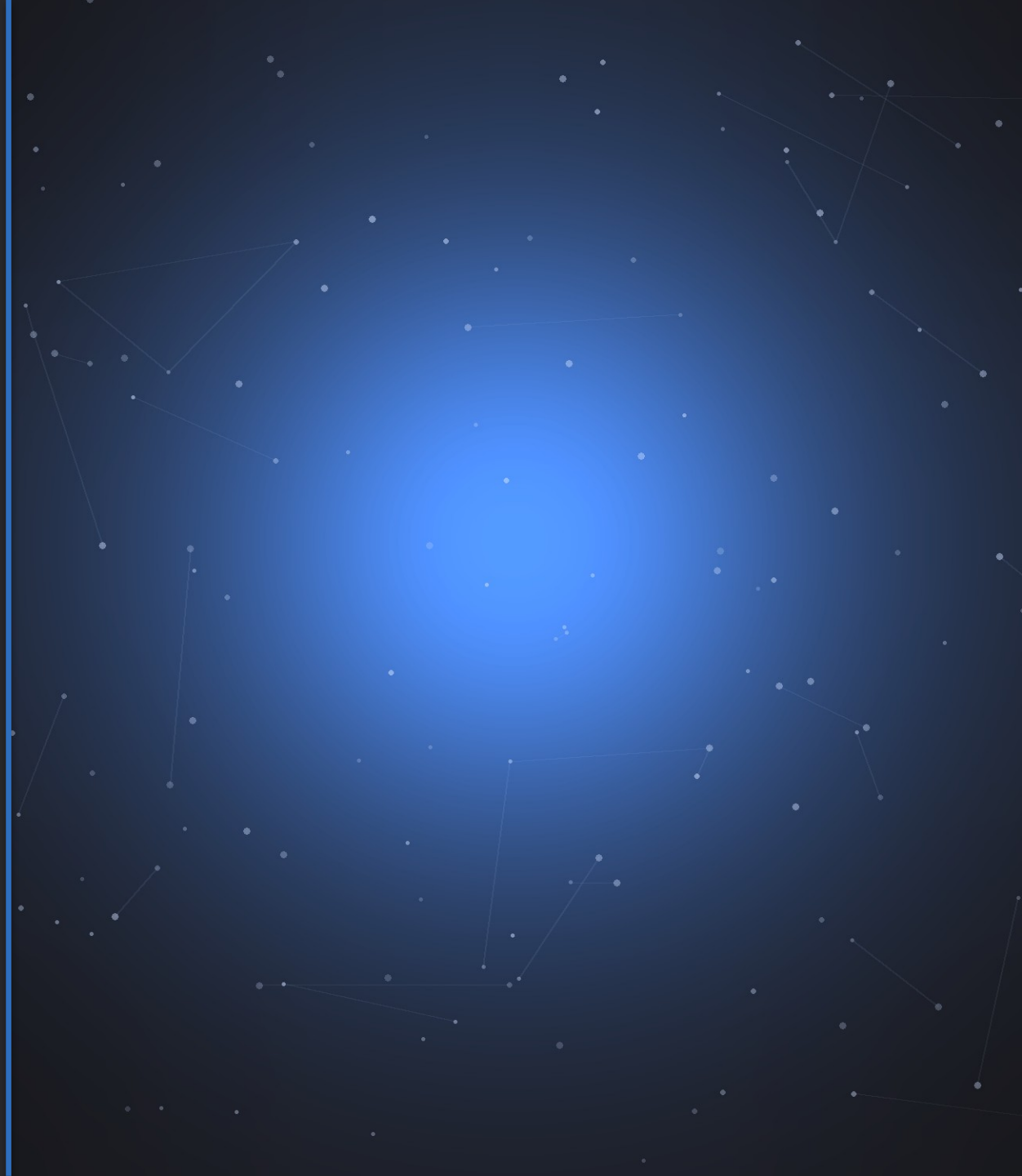

FLEX MODULES - PULL IN WHEN AHEAD

Claude Code Power Topics

Scott Hay

AIA Copilot



Six modules, deploy at will

01 **1. CLAUDE.md Power Patterns**
Day 1 flex - after Block 2 (Memory)

02 **2. The QoL Arsenal**
Day 1 flex - after Block 4 (Commands)

03 **3. Build a Custom MCP Server**
Day 3 flex - extends Block 1 (MCP)

04 **4. Code Execution & Tool Use**
Day 3 flex - extends Block 4 (Scale)

05 **5. The Agent SDK in 15 Minutes**
Day 3 flex - after Block 3 (Headless)

06 **6. The Wider Surface**
Any day - closer or energy break



FLEX MODULE 1 - DAY 1

CLAUDE.md Power Patterns

Pull in when ahead - pairs with Day 1 Block 2
(Memory & CLAUDE.md)

CONCEPT

Inside a lead engineer's file

Patterns from the CLAUDE.md of Claude Code's own creator - and teams shipping with it daily.



Lean - ~200 lines max - past that, rules get sampled, not followed



Behavioral - rules Claude can execute, not vibes it should absorb



Tagged - critical rules wrapped in `<xml_tags>` so they stand out

The self-improvement loop



to memory

Start a message with # - Claude files it into memory for you.



Correction becomes rule

Every time you fix Claude twice, write the rule once.



lessons.md

'After any correction, append the lesson' - as a standing rule.



Prune monthly

Dead rules erode trust in the live ones - delete them.



Litmus test: a new dev types 'run the tests' - if it works first try, your file is doing its job.

ADVANCED

HANDOFF.md: state that survives

The advanced form of 'state lives in files, not sessions'.



What it is

A short file: what changed, what's next, what's blocked.



The ritual

Last act of a session: write it.
First act of the next: read it.



The payoff

A fresh session is productive in one prompt, not ten.



Pairs with Day 1's 'state lives in files' principle - this is that principle made into a habit.

Paste this: teach it a lesson

What you are practicing

- Filing a rule without breaking flow
- Turning a correction into permanence
- Ending a session with a handoff

PASTE THIS

1

Always run the full test suite before declaring any task done

2

After any correction from me, append the lesson to lessons.md as a one-line rule.

3

Update HANDOFF.md: what changed, what's next, what's blocked. Keep it under 30 lines.



FLEX MODULE 2 - DAY 1

The QoL Arsenal

Pull in when ahead - pairs with Day 1 Block 4
(Commands, Cost & Permissions)

Cockpit switches worth mapping



/keybindings

Remap everything - ~/.claude/keybindings.json, chords included.



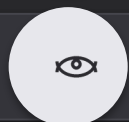
/statusline

A persistent HUD line: model, branch, context, elapsed time.



/rename + /resume

Label sessions like tickets - 'TODO: refactor auth' - and return later.



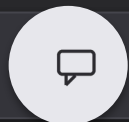
Ctrl+R

Search the full session transcript - find that command from an hour ago.



/terminal-setup

One-time fix so Shift+Enter inserts a newline in your terminal.



/btw

Ask a side question without polluting the main task context.



Weekly releases move these fast - re-verify each command's exact behavior on delivery morning.

/statusline: build your HUD

One persistent line at the bottom of every prompt.



What it shows

Model, git branch, context used - whatever you script.



Elapsed time

A running session timer keeps long tasks honest.



How you build it

Describe it in English - Claude writes the script itself.



The statusline is just a script in your settings - ask Claude to write and wire it for you.

Terminal moves that compound

MOVE	PAYOFF
Esc Esc	Jump back in the conversation - the rewind entry point
! prefix	Run a shell command; the output lands in context, no copy-paste
Tab	Complete file paths - stop typing them from memory
Paste an image	Screenshots go straight into the prompt - point at the bug
Shift+Tab	Cycle permission modes without leaving the keyboard

Paste this: rig the cockpit

What you are practicing

- A HUD you designed in English
- Session names that read like tickets
- Letting Claude configure Claude

PASTE THIS

1

/statusline show the model, current
git branch, context % used, and
minutes elapsed

2

/rename flex-lab-TODO-cleanup

3

/keybindings



FLEX MODULE 3 - DAY 3

Build a Custom MCP Server

Pull in when ahead - restores the build-your-own content cut from Day 3 Block 1

CONCEPT

From consumer to builder

A server is small: tools, descriptions, and a transport.



Tools - functions with model-facing descriptions



FastMCP - the Python decorator pattern - a tool in
~10 lines



Transport - stdio for local, Streamable HTTP for
shared

FastMCP in five steps

01

Install

Official MCP
SDK, fresh
venv.



02

Wrap

A plain
function as
a @tool.



03

Explain

Docstring
for the
model.



04

Run

Serve over
stdio.



05

Add

claude mcp
add
<name>

Tool design: the builder's bar



Task-shaped, not endpoint-shaped

One tool per job-to-be-done, not one per API route.



Descriptions for the model

Say when to use it, what comes back, what the limits are.



Errors that teach

'Order not found - IDs look like ORD-12345' beats a 404.



Paginate big results

Never let one call dump 10,000 rows into context.



The over-tooling trap

40 thin tools confuse the model and bloat every prompt.



When NOT to build: if a one-off script, a skill, or an existing server covers it - every tool costs context.

LIVE

Demo: Inspector on your server



LIVE DEMO

- 1 `npx @modelcontextprotocol/inspector python server.py`
- 2 Connect - list tools - read the description
- 3 Run `get_order_status` with a test ID
- 4 Break it: send a bad ID, read the error

Paste this: scaffold a server

What you are practicing

- Claude writes the boilerplate
- You own the tool descriptions
- Inspector before integration

PASTE THIS

1

Create a Python MCP server using the official SDK's FastMCP pattern with one tool: `get_order_status(order_id: str)`. Fake the data. Write the docstring for the model: when to use it, what returns, ID format.

2

```
npx @modelcontextprotocol/inspector  
python server.py
```

3

```
claude mcp add orders -- python server.py
```



FLEX MODULE 4 - DAY 3

Code Execution & Tool Use

Pull in when ahead - restores the advanced tool-use content cut from Day 3 Block 4

Round-trips or one script?



Traditional tool calls

- Every result round-trips through context
- All schemas ride along up front
- Model orchestrates step by step



Code execution

- Model writes code that calls tools
- Intermediates stay in the sandbox
- Only the final answer returns



Same tools, same job - the difference is where the intermediate data lives and who pays for it.

Three pillars of advanced use

Three mechanisms, one goal: stop paying for what the model doesn't need.



Tool search

Discover and load tool schemas on demand, not all up front.



Programmatic calling

Invoke tools from code in a sandbox - loop, filter, join there.



Tool examples

Show worked calls so the model nails tricky parameter shapes.



Sourced from Anthropic engineering posts - names and availability shift; verify current status at delivery.

The token math



The schema tax

Dozens of connected tools = tens of thousands of tokens before work starts.



The round-trip tax

Every intermediate result is billed through context - twice.



The sandbox dividend

Filter 10,000 rows in code; only the summary rides home.



Reported gains

Anthropic's published example: ~150K tokens down to ~2K.



Published benchmark figures, not your workload - measure your own before promising savings.

Paste this: think in scripts

What you are practicing

- Context as a decision bus, not a data bus
- Asking for scripts over call chains
- Summaries home, intermediates stay

PASTE THIS

1

Write and run a script that pulls the last 50 CI runs, filters to failures, and returns ONLY a summary table. Do not stream raw logs into the conversation.

2

Before you call tools one at a time, tell me: could a single script do this whole chain? If yes, write it.



FLEX MODULE 5 - DAY 3

The Agent SDK in 15 Minutes

Pull in when ahead - pairs with Day 3 Block 3
(Production & CI/CD)

CONCEPT

Claude Code as a library

The same agentic loop you have used for three days - importable.



Same loop - gather context - act - verify, now inside your process



Python & TypeScript - official packages, same options as the CLI



Your guardrails - tools, permissions, and hooks set in code

Stay on -p, or graduate?

✓ Stay with claude -p

- One-shot jobs and CI steps
- Shell-native glue is enough
- Humans read the output

🚀 Graduate to the SDK

- Custom tools run in-process
- Multi-turn state your app owns
- The agent IS the product

📘 Rule of thumb: the moment you're parsing -p output to make decisions in code, you wanted the SDK.

Paste this: hello, agent

What you are practicing

- Install is one line per language
- `query()` mirrors `claude -p`
- CLI knowledge transfers whole

PASTE THIS

1

```
pip install claude-agent-sdk
```

2

```
from claude_agent_sdk import query
async for msg in
query(prompt="Summarize this repo's
state"):
    print(msg)
```

3

```
npm install @anthropic-ai/claude-agent-sdk
```

Know your meter

Three billing paths - do not discover which one you're on in production.



Subscription

Interactive CLI use rides your Claude plan.



API / cloud

Per-token billing - the production default.



SDK plan credit

Eligible paid plans get a separate monthly Agent SDK credit.



The SDK credit is a separate MONTHLY credit, NOT a weekly pool - sizing/eligibility vary by plan, confirm the support doc at delivery.



FLEX MODULE 6 - ANY DAY

The Wider Surface

Pull in any day - works as a closer, an energy break, or a preview of what's next

CONCEPT

One account, many surfaces

The same Claude, reachable from more places than your terminal.



Artifacts - live interactive pages built from a prompt



Desktop + browser - the app, with a built-in browsing surface



Embedded - Chrome, Slack, and Cowork sessions

Artifacts: apps from prompts

Describe an app; get a running page.



Build

A working interactive page from one good prompt.



Share

Starts private; sharing follows plan and admin policy.



Connector-backed

Can call MCP connectors with the viewer's permissions.



Available on Pro, Max, Team, and Enterprise; sharing is admin-gated - verify your tenant before demoing.

The rest, one beat each



Remote Control

Continue a local session from phone or browser - research preview.



Trusted Devices

Beta - enrolled devices plus biometric or passkey confirmation.



Desktop + built-in browser

Claude outside the terminal, with a browsing surface.



Claude in Chrome

The agent acting inside your browser tabs.



Slack + Cowork

Claude in team chat; session-based collaborative work.



Every row is plan-, admin-, or preview-gated - demonstrations, not dependencies. Verify before showing.

Paste this: your first Artifact

What you are practicing

- Prompt-to-app in one shot
- Iterating on a live page
- Judging where Artifacts fit

PASTE THIS

1

Build me a retro-board web app: three columns - went well, needs work, actions - with sticky notes I can add and drag, and a button to export the board as markdown.

2

Now make it dark-themed and add a per-column note counter.

Back to the plan

What you added

A deeper cut than the syllabus promised - because the room earned the time.

Where it lives

Every module maps to a day and block - revisit it next to those notes.